



INTERNNOVA - JAVA INTERNSHIP

WEEK 4 - ASSIGNMENT



Submitted By: Subhasri S

Branch: Computer Science and Engineering

College: PSNA College of Engineering and Technology

Internship Domain: Java Development

Assignment: Week 4 – Exception Handling & Collections

Submission Date: 29 August 2026

OBJECTIVE & LEARNING OUTCOMES

The main objective of this assignment is to understand and implement Exception Handling and Collections in Java through practical programming tasks. The assignment focuses on handling runtime errors safely and managing groups of data efficiently using different collection classes.

The assignment covers important Week 4 concepts such as Exception Handling, Collections, HashSet, HashMap, and StringBuilder. These concepts are applied through individual programs and a Mini Java Project to develop practical programming skills.

The programs are designed to accept input, process the data, handle possible errors, and display meaningful results. This helps in understanding how Java applications can be made more reliable, flexible, and user-friendly.

The Mini Java Project combines the concepts covered in the individual tasks into a console-based Student Record Manager. It demonstrates how multiple Java features can work together to solve a practical problem.

Learning Outcomes

- Handle runtime errors using try-catch and appropriate exception classes.
- Use Collections to store, search, remove, and manage data efficiently.
- Use HashSet to maintain unique values without duplicate entries.
- Use HashMap to store and retrieve data using key-value pairs.
- Use StringBuilder for efficient string manipulation and output formatting.
- Combine multiple Week 4 concepts to develop a practical console-based application.

TASK 1: EXCEPTION HANDLING – SAFE CALCULATOR

Objective

To create a calculator program that performs basic arithmetic operations while safely handling division by zero and invalid user input.

Brief Explanation

This program accepts two numbers from the user and performs addition, subtraction, multiplication, division, and modulus operations. `ArithmeticException` is handled when the second number is zero, while `InputMismatchException` is used to handle invalid numeric input. Meaningful error messages are displayed so that the program does not terminate unexpectedly.

Source Code

File Name: `SafeCalculator.java`

```
import java.util.InputMismatchException;
import java.util.Scanner;

public class SafeCalculator {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter first number : ");
            double num1 = sc.nextDouble();

            System.out.print("Enter second number : ");
            double num2 = sc.nextDouble();

            System.out.println("\n=====");
            System.out.println("                CALCULATOR RESULTS");
            System.out.println("=====");

            System.out.println("Addition      : " + (num1 + num2));
            System.out.println("Subtraction   : " + (num1 - num2));
            System.out.println("Multiplication : " + (num1 * num2));

            try {
                if (num2 == 0) {
                    throw new ArithmeticException("Divisor cannot be zero");
                }

                System.out.println("Division      : " + (num1 / num2));
                System.out.println("Modulus       : " + (num1 % num2));

            } catch (ArithmeticException e) {
                System.out.println("Division      : " + e.getMessage());
                System.out.println("Modulus       : " + e.getMessage());
            }

        } catch (InputMismatchException e) {
            System.out.println("\nError: Please enter valid numeric values.");
        }

        sc.close();
    }
}
```

Output

```
● PS D:\Internova\Week_4> javac SafeCalculator.java
● PS D:\Internova\Week_4> java SafeCalculator.java
Enter first number : 10
Enter second number : 5

=====
                CALCULATOR RESULTS
=====
Addition      : 15.0
Subtraction   : 5.0
Multiplication : 50.0
Division      : 2.0
Modulus       : 0.0
```

Exception Handling Explanation

ArithmeticException is used to handle division or modulus by zero. InputMismatchException is used to handle invalid numeric input. The try-catch blocks allow the program to display meaningful error messages instead of terminating unexpectedly.

TASK 2: COLLECTIONS – STUDENT NAMES

Objective

To use a suitable Java Collection to store student names and perform operations such as adding, displaying, searching, and removing student names.

Brief Explanation

This program uses an ArrayList to store the names of five students. The collection allows names to be stored dynamically and maintains their insertion order. Student names are added and displayed using a loop. The remove() method is used to remove a student, while contains() is used to search for a particular student.

Source Code

File Name: StudentNames.java

```
import java.util.ArrayList;
import java.util.Scanner;

public class StudentNames {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        ArrayList<String> students = new ArrayList<>();

        System.out.println("Enter names of 5 students:");

        for (int i = 0; i < 5; i++) {
            System.out.print("Student " + (i + 1) + " : ");
            students.add(sc.nextLine());
        }

        System.out.println("\n=====");
        System.out.println("          STUDENT COLLECTION");
        System.out.println("=====");

        System.out.println("\nAll Student Names");
        System.out.println("-----");

        for (String student : students) {
            System.out.println(student);
        }

        System.out.print("\nEnter student name to remove : ");
        String removeName = sc.nextLine();

        if (students.remove(removeName)) {
            System.out.println("Student removed successfully.");
        } else {
            System.out.println("Student not found.");
        }

        System.out.print("\nEnter student name to search : ");
        String searchName = sc.nextLine();

        if (students.contains(searchName)) {
            System.out.println("Student found.");
        } else {
```

```

        System.out.println("Student not found.");
    }

    System.out.println("\nFinal Collection");
    System.out.println("-----");

    for (String student : students) {
        System.out.println(student);
    }

    sc.close();
}
}

```

Output

```

Module 5 : 0.0
● PS D:\Internova\Week_4> javac StudentNames.java
● PS D:\Internova\Week_4> java StudentNames
Enter names of 5 students:
Student 1 : Subhasri S
Student 2 : Sathish Kumar M
Student 3 : Vasundra M
Student 4 : Shabana Banu S
Student 5 : Sivaranjani S

=====
STUDENT COLLECTION
=====

All Student Names
-----
Subhasri S
Sathish Kumar M
Vasundra M
Shabana Banu S
Sivaranjani S

Enter student name to remove : Sivaranjani S
Student removed successfully.

Enter student name to search : Subhasri S
Student found.

Final Collection
-----

```

```

=====
STUDENT COLLECTION
=====

All Student Names
-----
Subhasri S
Sathish Kumar M
Vasundra M
Shabana Banu S
Sivaranjani S

Enter student name to remove : Sivaranjani S
Student removed successfully.

Enter student name to search : Subhasri S
Student found.

Final Collection
-----
Subhasri S
Sathish Kumar M
Vasundra M
Shabana Banu S

```

Collection Used: ArrayList

ArrayList is a Java Collection that stores elements dynamically and maintains insertion order. It provides methods such as add(), remove(), and contains() for managing student names.

TASK 3: HASHSET – UNIQUE COURSES

Objective

To use HashSet to store course names and demonstrate that duplicate course names are not stored.

Brief Explanation

This program uses a HashSet to store course names. Seven course entries are accepted from the user, including duplicate values. Since HashSet does not allow duplicate elements, repeated course names are automatically ignored. The final HashSet contains only unique course names. The program also allows the user to search for a particular course.

Source Code

File Name: UniqueCourses.java

```
import java.util.HashSet;
import java.util.Scanner;

public class UniqueCourses {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        HashSet<String> courses = new HashSet<>();

        System.out.println("Enter 7 course names:");

        for (int i = 0; i < 7; i++) {
            System.out.print("Course " + (i + 1) + " : ");
            courses.add(sc.nextLine());
        }

        System.out.println("\n=====");
        System.out.println("                UNIQUE COURSES");
        System.out.println("=====");

        System.out.println("\nFinal HashSet");
        System.out.println("-----");

        for (String course : courses) {
            System.out.println(course);
        }

        System.out.print("\nEnter course name to search : ");
        String searchCourse = sc.nextLine();

        if (courses.contains(searchCourse)) {
            System.out.println("Course found.");
        } else {
            System.out.println("Course not found.");
        }

        System.out.println("\nTotal Unique Courses : " + courses.size());

        sc.close();
    }
}
```

Output

```
● PS D:\Internova\Week_4> javac UniqueCourses.java
● PS D:\Internova\Week_4> java UniqueCourses
Enter 7 course names:
Course 1 : AIML
Course 2 : Computer Networks
Course 3 : Physics
Course 4 : Chemistry
Course 5 : Data Structures
Course 6 : Foundation of DataScience
Course 7 : Knowledge Engineering

=====
                UNIQUE COURSES
=====

Final HashSet
-----
Computer Networks
Chemistry
Data Structures
Foundation of DataScience
AIML
Knowledge Engineering
Physics

Enter course name to search : Chemistry
Course found.

Total Unique Courses : 7
```

Why HashSet Does Not Store Duplicates

HashSet stores only unique elements. If an equivalent value is already present, adding it again does not create another entry. Therefore, duplicate course names are automatically ignored.

TASK 4: HASHMAP – STUDENT RECORDS

Objective

To use HashMap to store student records using Student ID as the key and Student Name as the value.

Brief Explanation

This program uses a HashMap to maintain student records in key-value pairs. Student ID is used as the key and Student Name is stored as the corresponding value. The program adds five student records, displays all records, searches for a student using Student ID, and removes a student record. The updated records are displayed after the removal operation.

Source Code

File Name: StudentRecords.java

```
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class StudentRecords {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        HashMap<Integer, String> students = new HashMap<>();

        System.out.println("Enter details of 5 students:");

        for (int i = 0; i < 5; i++) {

            System.out.print("Student ID   : ");
            int id = sc.nextInt();
            sc.nextLine();

            System.out.print("Student Name : ");
            String name = sc.nextLine();

            students.put(id, name);
            System.out.println();
        }

        System.out.println("=====");
        System.out.println("          STUDENT RECORDS");
        System.out.println("=====");

        System.out.println("\nAll Student Records");
        System.out.println("-----");

        for (Map.Entry<Integer, String> entry : students.entrySet()) {
            System.out.println("Student ID   : " + entry.getKey());
            System.out.println("Student Name : " + entry.getValue());
            System.out.println();
        }

        System.out.print("Enter Student ID to search : ");
        int searchId = sc.nextInt();
```

```

        if (students.containsKey(searchId)) {
            System.out.println("Student Name : " + students.get(searchId));
        } else {
            System.out.println("Student record not found.");
        }

        System.out.print("\nEnter Student ID to remove : ");
        int removeId = sc.nextInt();

        if (students.containsKey(removeId)) {
            students.remove(removeId);
            System.out.println("Student record removed successfully.");
        } else {
            System.out.println("Student record not found.");
        }

        System.out.println("\nUpdated Student Records");
        System.out.println("-----");

        for (Map.Entry<Integer, String> entry : students.entrySet()) {
            System.out.println("Student ID   : " + entry.getKey());
            System.out.println("Student Name : " + entry.getValue());
            System.out.println();
        }

        sc.close();
    }
}

```

Output

```

PS D:\Internova\Week_4> javac StudentRecords.java
PS D:\Internova\Week_4> java StudentRecords
Enter details of 5 students:
Student ID   : 1
Student Name : Subhasri

Student ID   : 2
Student Name : Vasundra

Student ID   : 3
Student Name : Sivaranjani

Student ID   : 4
Student Name : Shabanabanu

Student ID   : 5
Student Name : Vaishnavi

=====
          STUDENT RECORDS
=====

All Student Records
-----
Student ID   : 1
Student Name : Subhasri

Student ID   : 2
Student Name : Vasundra

```

```

Student ID   : 3
Student Name : Sivaranjani

Student ID   : 4
Student Name : Shabanabanu

Student ID   : 5
Student Name : Vaishnavi

Enter Student ID to search : 1
Student Name : Subhasri

Enter Student ID to remove : 5
Student record removed successfully.

Updated Student Records
-----
Student ID   : 1
Student Name : Subhasri

Student ID   : 2
Student Name : Vasundra

Student ID   : 3
Student Name : Sivaranjani

Student ID   : 4
Student Name : Shabanabanu

```

HashMap Explanation

HashMap stores data in key-value pairs. Each key is unique and is used to access its corresponding value. In this program, Student ID is the key and Student Name is the value.

TASK 5: STRINGBUILDER – TEXT MANIPULATION

Objective

To use StringBuilder in Java to perform append, insert, replace, and reverse operations on a string.

Brief Explanation

This program demonstrates the use of StringBuilder for modifying text. A string is created first, then additional text is appended. New text is inserted at a specified position and a portion of the text is replaced. Finally, the complete string is reversed and the result is displayed after each operation.

Source Code

File Name: TextManipulation.java

```
public class TextManipulation {  
    public static void main(String[] args) {  
        StringBuilder text = new StringBuilder("Java Programming");  
  
        System.out.println("=====");  
        System.out.println("          STRINGBUILDER DEMO");  
        System.out.println("=====");  
  
        System.out.println("\nOriginal String");  
        System.out.println("-----");  
        System.out.println(text);  
  
        text.append(" is Easy");  
  
        System.out.println("\nAfter Append");  
        System.out.println("-----");  
        System.out.println(text);  
  
        text.insert(5, "Object Oriented ");  
  
        System.out.println("\nAfter Insert");  
        System.out.println("-----");  
        System.out.println(text);  
  
        text.replace(5, 20, "OOP");  
  
        System.out.println("\nAfter Replace");  
        System.out.println("-----");  
        System.out.println(text);  
  
        text.reverse();  
  
        System.out.println("\nAfter Reverse");  
        System.out.println("-----");  
        System.out.println(text);  
    }  
}
```

Output

```
● PS D:\Internova\Week_4> javac TextManipulation.java
● PS D:\Internova\Week_4> java TextManipulation
=====
                STRINGBUILDER DEMO
=====

Original String
-----
Java Programming

After Append
-----
Java Programming is Easy

After Insert
-----
Java Object Oriented Programming is Easy

After Replace
-----
Java OOP Programming is Easy

After Reverse
-----
ysaE si gnimmargorP POO avaJ
```

StringBuilder Explanation

StringBuilder is used to create and modify a mutable sequence of characters. It provides methods such as `append()`, `insert()`, `replace()`, and `reverse()` for efficient text manipulation.

TASK 6: MINI JAVA PROJECT – STUDENT RECORD MANAGER

Objective

To develop a console-based Student Record Manager by combining Exception Handling, Collections, HashSet, HashMap, and StringBuilder concepts in one practical Java application.

Brief Explanation

The Student Record Manager is a menu-driven application for managing student information. It allows the user to add, view, search, and remove student records. HashMap is used to store student information, while HashSet is used to maintain unique course names. Exception handling is used for invalid menu choices and numeric input, and StringBuilder is used to construct formatted student information.

Source Code

File Name: StudentRecordManager.java

Main Features

- Add Student – Accepts Student ID, Student Name, Course, and Age.
- View Students – Displays all stored student records.
- Search Student – Searches for a student using Student ID.
- Remove Student – Removes a student record using Student ID.
- Display Unique Courses – Displays unique course names using HashSet.
- Exception Handling – Handles invalid menu choices and numeric input.
- StringBuilder – Used to format student information and output.

```
import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Scanner;

public class StudentRecordManager {

    static HashMap<Integer, String> studentNames = new HashMap<>();
    static HashMap<Integer, String> studentCourses = new HashMap<>();
    static HashMap<Integer, Integer> studentAges = new HashMap<>();
    static HashSet<String> courses = new HashSet<>();

    static Scanner sc = new Scanner(System.in);

    public static void addStudent() {

        try {
            System.out.print("Enter Student ID   : ");
            int id = sc.nextInt();
            sc.nextLine();

            if (studentNames.containsKey(id)) {
                System.out.println("Error: Student ID already exists.");
                return;
            }
        }
```

```

        System.out.print("Enter Student Name : ");
        String name = sc.nextLine();

        System.out.print("Enter Course      : ");
        String course = sc.nextLine();

        System.out.print("Enter Age        : ");
        int age = sc.nextInt();

        if (age <= 0) {
            System.out.println("Error: Invalid age.");
            return;
        }

        studentNames.put(id, name);
        studentCourses.put(id, course);
        studentAges.put(id, age);
        courses.add(course);

        System.out.println("Student added successfully.");

    } catch (java.util.InputMismatchException e) {
        System.out.println("Error: Please enter valid input.");
        sc.nextLine();
    }
}

public static void viewStudents() {

    if (studentNames.isEmpty()) {
        System.out.println("No student records available.");
        return;
    }

    System.out.println("\n=====");
    System.out.println("          STUDENT RECORDS");
    System.out.println("=====");

    for (Map.Entry<Integer, String> entry : studentNames.entrySet()) {

        int id = entry.getKey();

        StringBuilder details = new StringBuilder();

        details.append("Student ID   : ").append(id).append("\n");
        details.append("Student Name : ").append(entry.getValue()).append("\n");
        details.append("Course      : ").append(studentCourses.get(id)).append("\n");
        details.append("Age        : ").append(studentAges.get(id)).append("\n");

        System.out.println(details);
    }
}

public static void searchStudent() {

    try {
        System.out.print("Enter Student ID to search : ");
        int id = sc.nextInt();

        if (studentNames.containsKey(id)) {

            StringBuilder details = new StringBuilder();

            details.append("\nStudent Found\n");
            details.append("-----\n");
            details.append("Student ID   : ").append(id).append("\n");

```

```

        details.append("Student Name : ").append(studentNames.get(id)).append("\n");
        details.append("Course       : ").append(studentCourses.get(id)).append("\n");
        details.append("Age         : ").append(studentAges.get(id)).append("\n");

        System.out.println(details);

    } else {
        System.out.println("Student not found.");
    }

} catch (java.util.InputMismatchException e) {
    System.out.println("Error: Student ID must be a number.");
    sc.nextLine();
}

}

public static void removeStudent() {

    try {
        System.out.print("Enter Student ID to remove : ");
        int id = sc.nextInt();

        if (studentNames.containsKey(id)) {

            studentNames.remove(id);
            studentCourses.remove(id);
            studentAges.remove(id);

            System.out.println("Student removed successfully.");

        } else {
            System.out.println("Student not found.");
        }

    } catch (java.util.InputMismatchException e) {
        System.out.println("Error: Student ID must be a number.");
        sc.nextLine();
    }

}

public static void displayUniqueCourses() {

    System.out.println("\n=====");
    System.out.println("        UNIQUE COURSES");
    System.out.println("=====");

    if (courses.isEmpty()) {
        System.out.println("No courses available.");
        return;
    }

    for (String course : courses) {
        System.out.println(course);
    }

}

public static void main(String[] args) {

    int choice = 0;

    do {

        System.out.println("\n=====");
        System.out.println("        STUDENT RECORD MANAGER");
        System.out.println("=====");

        System.out.println("1. Add Student");

```

```

        System.out.println("2. View Students");
        System.out.println("3. Search Student");
        System.out.println("4. Remove Student");
        System.out.println("5. Display Unique Courses");
        System.out.println("6. Exit");

        System.out.print("\nEnter your choice : ");

        try {
            choice = sc.nextInt();
            sc.nextLine();

            switch (choice) {

                case 1:
                    addStudent();
                    break;

                case 2:
                    viewStudents();
                    break;

                case 3:
                    searchStudent();
                    break;

                case 4:
                    removeStudent();
                    break;

                case 5:
                    displayUniqueCourses();
                    break;

                case 6:
                    System.out.println("Thank you for using Student Record Manager.");
                    break;

                default:
                    System.out.println("Error: Invalid menu choice.");
            }

        } catch (java.util.InputMismatchException e) {
            System.out.println("Error: Please enter a valid menu number.");
            sc.nextLine();
        }

    } while (choice != 6);

    sc.close();
}

```


Output

```
PS D:\Internova\Week_4> javac StudentRecordManager.java
PS D:\Internova\Week_4> java StudentRecordManager

=====
STUDENT RECORD MANAGER
=====
1. Add Student
2. View Students
3. Search Student
4. Remove Student
5. Display Unique Courses
6. Exit

Enter your choice : 1
Enter Student ID : 1
Enter Student Name : Subhasri S
Enter Course : Computer Science
Enter Age : 19
Student added successfully.

=====
STUDENT RECORD MANAGER
=====
1. Add Student
2. View Students
3. Search Student
```

```
3. Search Student
4. Remove Student
5. Display Unique Courses
6. Exit
```

Enter your choice : 2

```
=====
STUDENT RECORDS
=====
Student ID : 1
Student Name : Subhasri S
Course : Computer Science
Age : 19
```

```
=====
STUDENT RECORD MANAGER
=====
1. Add Student
2. View Students
3. Search Student
4. Remove Student
5. Display Unique Courses
6. Exit
```

Enter your choice : 3
Enter Student ID to search : 1

Student Found

Enter your choice : 3
Enter Student ID to search : 1

Student Found

```
-----
Student ID : 1
Student Name : Subhasri S
Course : Computer Science
Age : 19
```

```
=====
STUDENT RECORD MANAGER
=====
1. Add Student
2. View Students
3. Search Student
4. Remove Student
5. Display Unique Courses
6. Exit
```

Enter your choice : 5

```
=====
UNIQUE COURSES
=====
Computer Science
```

```
=====
STUDENT RECORD MANAGER
=====
```

```
1. Add Student
2. View Students
3. Search Student
4. Remove Student
5. Display Unique Courses
6. Exit
```

Enter your choice : 4
Enter Student ID to remove : 1
Student removed successfully.

```
=====
STUDENT RECORD MANAGER
=====
1. Add Student
2. View Students
3. Search Student
4. Remove Student
5. Display Unique Courses
6. Exit
```

Enter your choice : 6
Thank you for using Student Record Manager.

Concepts Used in the Mini Project

Exception Handling – Handles invalid input and runtime errors.

Collections – Used to store and manage student data.

HashMap – Stores student records using Student ID.

HashSet – Stores unique course names.

StringBuilder – Formats student information and output.